# Zero-Day Attack Detection System Using Autoencoders and Isolation Forest: An Unsupervised Machine Learning Approach

Mujeeb Ur Rehman[1 [0000−0002−4228−385X]], Margaret Zita[1 [0009−0005−3245−7210]], Muhammad Abrar[1 [0009−0001−5494−7529]], Muhammad Kazim[1 [0000−0001−8621−033X]], and Sohail Khalid[2 [0000−0003−3907−0236]]

[1] School of Computer Science and Informatics, De Montfort University, Leicester, UK
mujeeb.rehman@dmu.ac.uk p2889546@my365.dmu.ac.uk p2849363@my365.dmu.ac.uk muhammad.kazim@dmu.ac.uk

[2] Electrical and Computer Engineering Department, Riphah International University, Islamabad, Pakistan
s.khalid@riphah.edu.pk

**Abstract.** Detecting zero-day attacks remains a critical challenge in cybersecurity because they exploit previously unknown vulnerabilities, rendering traditional security tools that rely on known signature patterns ineffective. In this study, we present an unsupervised deep learning (DL) detection system that uses an autoencoder to effectively capture the zero-day attacks. The main objective was to build a system with a high recall and low false-negative rates. We used the CIC-IDS2017 dataset for the evaluation. We compared the results of our model with those of the Isolation Forest to demonstrate the efficacy of our model. The results indicate that autoencoders are effective for detecting zero-day attacks. The proposed system exhibited an accuracy of 98.9%, low false negatives, and scalable potential for real-world deployment. This study contributes a robust and adaptable framework for next-generation zero-day detection systems.

**Keywords:** Zero-day attack · Intrusion Detection · Deep learning · Autoencoders · Isolation forest

## 1 Introduction

In recent years, there has been an increase in cyberattacks, among which zero-day attacks remain some of the most difficult to defend against and are particularly destructive in nature. Zero-day attacks occur when hackers exploit flaws in computer software, hardware, or firmware before developers can detect them [3]. Fig. 1 shows a zero-day vulnerability timeline with vulnerability events from discovery to patching or mitigation. These events include vulnerability discovery, exploitation, public disclosure, and patching. The fact that this is a multistep process emphasizes the sophistication and covert nature of zero-day attacks, as


**Fig. 1.** Zero-Day Attack Life Cycle

well as the need for advanced detection skills beyond conventional security controls. Zero-day attacks can cause severe damage to organizations, governments, and individuals by stealing data, disrupting services, and damaging their reputations. This typically leads to financial losses and threatens the national security. Fig. 2 zero-day vulnerabilities exploited in the wild, with 75 zero-day vulnerabilities exploited in 2024, down from 98 the previous years [16]. Although 2024 shows a decrease from 2023, the total remains high compared to the pre-2021 period.

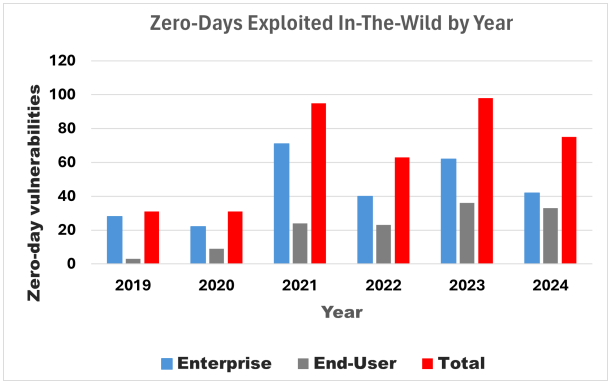


**Fig. 2.** Zero-Days Exploited In-The-Wild by Year [5]

Traditional machine learning (ML) methods are ineffective for detecting zero-day attacks because they rely on known signatures. However, zero-day attacks are novel and previously unseen, and their signatures are not available in training data. Consequently, the primary research focus has shifted to unsupervised deep learning (DL) approaches for anomaly detection. To better detect zero-day attacks, we propose an unsupervised DL autoencoder (AE).

AEs are unsupervised algorithms that learn relationships and patterns using unlabelled datasets. They reconstructed the input data using encoders to compress the data into a lower-dimensional latent space and decoders to reconstruct the original input from the latent-space representation. For anomaly detection, autoencoders are trained on normal data such that when an anomalous input, such as a zero-day attack, is provided, it produces a high reconstruction error, indicating a potential threat.

The architecture of the autoencoder is illustrated in Fig. 5. The encoder compresses the data into a lower-dimensional latent space, and the decoder uses the latent representation to recover the original input [20].

Traditional signature-based intrusion detection systems rely on known sets of rules and attack patterns [17]. Although they are effective against known attacks, they are unreliable in detecting zero-day ones. This gap has drawn more attention to anomaly based detection to identify deviations from normal network activities and unknown attacks, such as zero-day attacks. [14]. Unlike traditional methods, these anomalies can be identified using unsupervised and semi-supervised deep learning methods that do not require large amounts of labelled data. Autoencoders, recurrent neural networks, and convolutional neural networks have been found to be highly efficient in detecting anomalies [1].

Zero-day attack detection has increasingly leveraged deep learning methods owing to their generalization capabilities and ability to detect unknown threats. Hindy et al. used an auto-encoder-based model trained on the NSL-KDD dataset and achieved 92.96% accuracy, demonstrating the efficacy of neural network-motivated architectures for zero-day attack detection [8]. Kim et al. applied a transferred deep convolutional generational adversarial network (tDCGAN) to a malware dataset and achieved 95.74% accuracy, thereby demonstrating the strength of GAN-based models in handling zero-day threats [12].

Kumar and Sinha presented a hybrid heuristic (H-H) algorithm, which was tested on the CICIDS18 dataset and achieved an accuracy of 91.62%, due to the necessity for intelligence and robustness in detection systems [13]. Hairab et al. also used Convolutional Neural Networks (CNN) with regularization techniques on the TON-IoT dataset, achieving a 98% high detection accuracy and replicating the success of deep CNNs for anomaly detection tasks [9]. Zhao et al. used transfer learning to detect unknown attacks. Their results showed an improvement in the performance of detecting unknown attacks compared with the baselines [22]. Edakkat and Ajay proposed a hybrid ensemble model combining XGBoost and AdaBoost classifiers, which achieved an accuracy of 82%, outperforming individual classifiers such as XGBoost (75%), Random Forest (77%),

and AdaBoost (41%) in detecting zero-day attacks [4]. These studies refer to the increasing application of deep learning models to resolve these attacks.

The Canadian Institute for Cybersecurity has identified CIC-IDS2017 as a benchmark dataset for training and validating models in the field of cybersecurity. This dataset is well-known for its realistic flow-based network traffic, including several attack methods in a controlled environment [10]. This makes it suitable for evaluating deep learning models.

This literature review highlights several challenges of existing zero-day detection approaches: limited dataset diversity, dependence on labelled attack data, and the requirement for manual feature engineering. Many traditional and deep learning models also struggle to adapt to unknown threats in situations where labelled malicious data are scarce or unavailable.

In contrast, our proposed autoencoder-based model effectively addresses the limitations listed in Table 1. By using unsupervised learning trained on benign data, the reliance on labelled attack data is removed. Zero-day attacks were detected using reconstruction error analysis. In addition, autoencoders automatically learn latent feature representations, thereby reducing the need for manual feature engineering to a minimum. These strengths make our approach a promising solution for improving the robustness and generalization of detection systems. The contributions of this study include the implementation of a robust and reliable autoencoder model for zero-day detection, the construction of an Isolation Forest (IF) for outlier detection, and a comparison of the performance of the proposed model with that of an IF.

Section II outlines the methodology, and the remainder of this paper is organized as follows. The findings and discussion are presented in Section III. Section IV concludes the paper, and Section V discusses possible future directions.

## 2 Methodology

The proposed methodology is based on the following phases: Dataset description, data preprocessing, model development, test data preparation, and evaluation. Fig. 3 illustrates the proposed methodology flow.

### 2.1 Dataset Description

The Canadian Institute for Cybersecurity identified the CIC-IDS2017 dataset [10] as a benchmark dataset for network intrusion detection (NID) purposes. With more than 2.8 million records of network flows, it distinguishes between normal (benign) and harmful (malicious) traffic types, such as Distributed Denial of Service (DDoS), port scanning, and web attacks [15]. The wide range of attack examples allows for a comprehensive evaluation of the detection methodologies [2]. Despite this, a significant disparity class imbalance [6] and attack data often comprise a much smaller percentage of the data than benign traffic data. For example, DDoS instances appear at a ratio of 1:5 against benign traffic, whereas web attacks occur at a ratio of 1:50. In addition, the dataset

**Table 1.** Comparative summary of zero-day attack detection approaches

| Ref | Techniques | Accuracy (%) | Key Points |
|---|---|---|---|
| [8] | Autoencoder | 75 – 99 | Demonstrated neural networks' strength in detecting zero-day attacks. |
| [12] | tDCGAN | 95.74 | GAN-based models effective for detecting zero-day threats. |
| [13] | H-H Algorithm | 91.62, 88.98 | Emphasized intelligence and robustness in detection systems using hybrid heuristics. |
| [9] | CNN | 83.15 | CNNs with regularization improve anomaly detection performance in cybersecurity. |
| [22] | Spectral transformation, KNN, SVM, DT | 70 | Transfer learning reuses knowledge from data-rich sources to enhance detection in environments with limited labelled. |
| [4] | AdaBoost, XGBoost, Random Forest | 82 | Ensemble methods outperform individual classifiers, increasing robustness and detection rates. |

captures various metrics at the network level,including packet size, flow duration, protocol type, and mean interpacket time. Owing to its ability to reflect actual events, this dataset is invaluable for evaluating the performance of the zero-day detection systems. Its adaptability to simulate real-world scenarios is particularly beneficial for exploring zero-day exploits, which may hide harmful behaviors that are undetected by conventional methods.

### 2.2 Data preprocessing

Data preprocessing was performed on the raw data to prepare them for further processing. The raw CIC-IDS2017 dataset network traffic was transformed into a data format suitable for unsupervised learning while preserving the signature attacks.

**Feature Extraction** The PCAP files were converted into bidirectional flow features using the CICFlowMeter tool. The tool captures 84 statistical, temporal, and behavioral attributes, as follows:

- **Temporal Features:** Flow duration, packet inter-arrival times, and active/idle periods.
- **Statistical Features:** Total forward/backward packets, byte counts, mean packet length, and standard deviation.
- **Behavioral Features:** Ratios of forward-to-backward packets, SYN/FIN flag counts, and flow entropy.

These features are protocol-agnostic and retain patterns that are critical for detecting encrypted attacks.

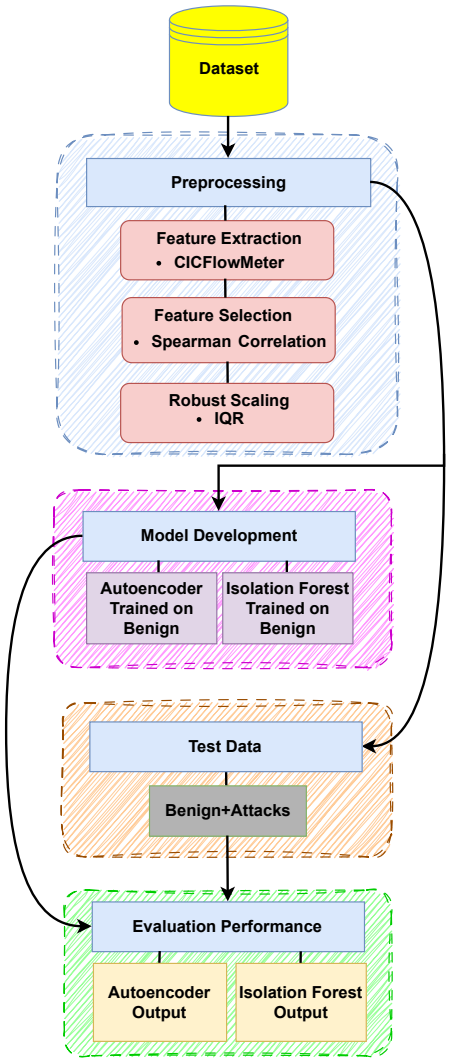


**Fig. 3.** Proposed Methodology

**Feature Selection** To remove redundancy, Spearman's rank correlation was used with a threshold of 0.85. Unlike the Pearson correlation, Spearman's method can capture nonlinear relationships that are common in network traffic, for example, between Flow Bytes and Total Packets. Features with correlations lower than the threshold were used for training, whereas those with high correlations were excluded from the training dataset.

**Robust Scaling** The features were normalized using interquartile range (IQR) scaling as follows:

$$z = \frac{x - Q_2(x)}{Q_3(x) - Q_1(x)}$$

where $Q_1(x)$, $Q_2(x)$, and $Q_3(x)$ denote the 25th, 50th (median), and 75th

**Data Splitting** To mimic real-world deployment, the dataset is chronologically divided into

- **Training (70%):** Benign traffic from the first day of the dataset.
- **Validation (15%):** Benign traffic from the subsequent day.
- **Testing (15%):** Remaining traffic, including zero-day attacks.

This partitioning ensured that the models were trained on normal data and evaluated using unseen attack patterns, thereby avoiding assumptions regarding the attack distribution.

### 2.3 Autoencoder-Based Model

The proposed autoencoder leverages the architecture shown in 4 to extract the latent representations of benign network traffic. Table II shows the training

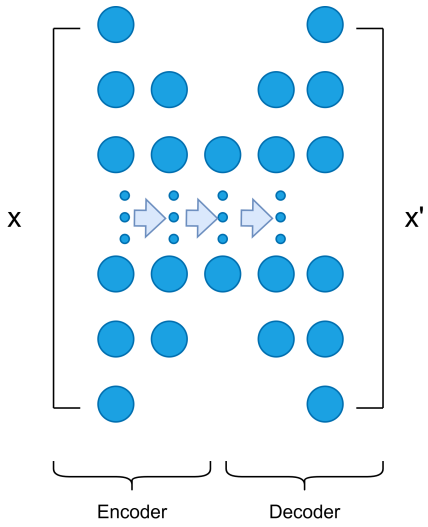


**Fig. 4.** Autoencoder architecture

and final architectures. Hyperparameters, including the layers, hidden neurons, epochs, and learning rate, were optimized using the random search method. The

random search focused on a common range of hyperparameters that also characterized the previous settings: depth of the encoder-decoder (2-4 layers), neurons per layer (16-128), learning rates (0.0001–0.005), and batch size (64-256). It was selected because it performs better than grid searching high-dimensional parameter spaces. It has a faster convergence than grid search methods in high-dimensional parameter spaces, and it can avoid overfitting by considering several parameter setups.

It was then trained on 75% of the benign samples once the best hyperparameters were determined, reserving 25% of the set for validation. The resulting final architecture (64→32→16 encoder layers with an 8-neurone bottleneck) and learning rate of 0.001 were determined through minimum validation-reconstruction error. Huber Loss ($\delta = 1.0$) was used instead of MSE because it dealt with outliers more effectively and fine-grained anomalies more efficiently. The generalization and training time were maximally balanced using a batch size of 128. The autoencoder was trained using the selected architecture for n epochs, where n was determined based on early stopping (patience = 10 epochs). The training progress was monitored using loss and accuracy curves to check for convergence of the model. The end-to-end training pipeline is presented in Algorithm 1.

The encoder-decoder model uses sequentially shrinking and expanding layers ($64 \rightarrow 32 \rightarrow 16 \rightarrow 8$ and vice versa) to learn the essential latent representations of the benign flows. The use of the Huber Loss enhances outlier pattern robustness, and a threshold-based anomaly scoring scheme ensures sensitivity to rare deviations. A schematic of the model is shown in Fig. 4. The end-to-end training procedure is described in Algorithm 1.

**Table 2.** Autoencoder Model Specifications

| **Component** | **Specification** |
|---|---|
| Encoder Layers | $64 \rightarrow 32 \rightarrow 16$ neurons (ReLU) |
| Latent Bottleneck | 8 neurons (Linear) |
| Decoder Layers | $16 \rightarrow 32 \rightarrow 64$ neurons (ReLU) |
| Output Layer | 84 neurons (Sigmoid) |
| Learning Rate | 0.001 |
| Batch Size | 128 |
| Loss Function | Huber Loss ($\delta = 1.0$) |
| Early Stopping | Patience = 10 epochs |

The model was trained on 75% of the benign data after the ideal hyperparameters were determined, with the remaining 25% of the dataset set aside for validation. The autoencoder was initialized with the selected architecture and trained for $n$ epochs, where $n$ was determined by early stopping (patience = 10

epochs). The training progress was monitored using loss and accuracy curves to ensure that the model converged. The complete training workflow is presented in Algorithm 1.

**Algorithm 1** Autoencoder Training

**Input:** Benign training data $D_{\text{train}}$, validation data $D_{\text{val}}$
**Output:** Trained Autoencoder model $M_{\text{AE}}$
Perform random search to select optimal hyper-parameters:
- Number of layers
- Hidden units
- Learning rate
- Batch size
- Training epochs

Initialize the Autoencoder model with the selected hyper-parameters
**for** each epoch **do**
    Train the model on $D_{\text{train}}$ by minimizing Huber loss
    Evaluate the model on $D_{\text{val}}$
    **if** validation loss does not improve for a defined number of epochs (patience) **then**
        Stop training early
    **end if**
**end for**
**return** Trained Autoencoder model $M_{\text{AE}}$

### 2.4 Threshold-Based Anomaly Detection

After training, the autoencoder was evaluated for the attack instance. The reconstruction error threshold $\tau$ is derived from the validation set to distinguish between benign and malicious traffic. The complete anomaly detection process is formalized in Algorithm 2.

### 2.5 Isolation Forest Model

The IF model was implemented as a baseline model for unsupervised anomaly detection because of its computational efficiency and ability to effectively isolate zero-day attack patterns without requiring labelled attack data. It was trained on 75% of the benign data, with key parameters set to 100 trees and a subsample size of 256, as outlined in Algorithm 3.

These hyperparameters were determined a priori using the validation set performance. A multiple estimators equal to 100 was found to be a good balance between diversity in isolation paths and the computational burden. The size of the subsample was 256, which was sufficiently large to separate the outliers from the benign training set. Larger subsample sizes $> 512$ caused lower sensitivity to rare anomalies, and smaller sizes $> 512$ added a high variance to the anomaly

**Algorithm 2** Threshold-based Attack Detection

**Input:** Validation reconstruction errors $\{L_1, L_2, \ldots, L_N\}$
**Output:** Threshold $\tau$
Compute mean $\mu_L$ and standard deviation $\sigma_L$ of the reconstruction errors
Calculate threshold: $\tau = \mu_L + k \cdot \sigma_L$ ▷ $k$ is tunable, e.g., 2 or 3
**for** each test instance $x$ **do**
    Compute reconstruction error: Huber$(x, x')$
    **if** reconstruction error $> \tau$ **then**
        Classify $x$ as *attack*
    **else**
        Classify $x$ as *benign*
    **end if**
**end for**

scores. The final configuration had the best trade-off value, and it obtained the best F1 score and recall on the evaluation CIC-IDS 2017 and cross-validation.

It is also necessary to measure the effect of transfer learning among different domains or application scenarios, which would demonstrate the flexibility and adaptability of the approach for practical applications in the future.

### 2.6 Cross-Dataset Generalization Testing

To rigorously evaluate the generalization capability of the proposed unsupervised models, we performed cross-dataset testing using two additional benchmark datasets: UNSW-NB15 and CSE-CIC-IDS2018. The autoencoder and Isolation Forest models were both trained solely on benign data from CIC-IDS2017 and tested on attack samples from these external datasets without retraining or fine-tuning. This setting replicates a true zero-day scenario in which the model must identify novel attack patterns that have not been observed during training. Prior to testing, the two external datasets were pre-processed to conform to the CIC-IDS2017 feature format. The features were scaled using the same robust scaling (IQR-based normalization) applied during training to ensure that a uniform baseline existed. Feature mapping was also performed, where the feature names differed, and the missing features were discarded. Shared features across all datasets were used during inference. Table 3 summarizes within-dataset results on CIC-IDS2017; Table 4 reports cross-dataset generalization on UNSW-NB15 and CSE-CIC-IDS2018. For UNSW-NB15, we downloaded 40,000 attack and 80,000 benign flow data. For CSE-CIC-IDS2018, Day 3 attack instances and the respective benign flows (100,000 in total) were used in the training. As shown in Table 4, both models achieve high detection accuracy on unseen data, while Table 3 provides the within-dataset baseline. The autoencoder achieved over 95% precision on both datasets, whereas the Isolation Forest also had fair generalization against lower recall. This indicates the robustness of both solutions against out-of-distribution and domain-shifted attack traffic.

This cross-dataset testing concept is consistent with the rationale for unsupervised transfer learning. The models were trained only on benign traffic from

**Algorithm 3** IF Training and Anomaly Scoring

**Input:** Benign training data $D_{\text{train}}$, subsample size $\psi = 256$, number of trees $t = 100$
**Output:** Trained IF model $M_{\text{IF}}$
Initialize IF with $t$ trees and subsample size $\psi$
**for** $i = 1$ to $t$ **do**
    Randomly sample $\psi$ instances from $D_{\text{train}}$
    Build an isolation tree by recursively splitting features at random
**end for**
**for** each test instance $x$ **do**
    Compute the average path length $E(h(x))$ across all trees
    Compute anomaly score: $s(x) = 2^{-E(h(x))/c(\psi)}$
**end for**
**return** Trained IF model $M_{\text{IF}}$

CIC-IDS2017, and their performance on attacks from other datasets (UNSW-NB15 and CSE-CIC-IDS2018) was tested without retraining or adaptation. This mimics the scenario in practice, where a deployed system is required to identify anomalies in an evolving network setting or new domains without being informed of labelled attack data. The high effectiveness of both the encoder and isolation forest when deployed in this context (Table 4) validates their adaptability and robustness, which are two critical factors for actual use in the wild in dynamic cybersecurity applications.

**Table 3.** EVALUATION METRICS

| **Metric** | **Autoencoder** | **IF** |
|---|---|---|
| **Accuracy (%)** | 98.9 | 93.2 |
| **Precision (%)** | 98.3 | 92.1 |
| **Recall (%)** | 99.2 | 93.8 |
| **F1-Score (%)** | 98.8 | 92.9 |
| **False Positive Rate (%)** | 0.8 | 3.4 |

## 3 Results and Discussion

This section describes the performance of the proposed ZDA detection system. The IF model achieved an accuracy of 93.2%, whereas the proposed autoencoder model outperformed it with 98.9% accuracy as shown in Fig. 5

Table 3 provides a detailed comparison of both models on important assessment criteria, such as accuracy, precision, recall, F1-score, and false positive rate, demonstrating the resilience of the autoencoder in detecting zero-day attacks.

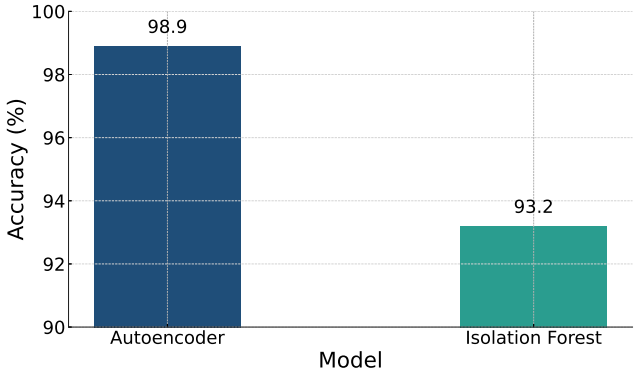


**Fig. 5.** Comparison of obtained accuracies

The performance comparison between the proposed system and different studies for zero-day attack detection is listed in Table 5

**Table 4.** Cross-Dataset Generalization Results

| Model | Dataset | Accuracy (%) | Precision (%) | Recall (%) | F1-Score (%) |
|---|---|---|---|---|---|
| Autoencoder | CIC-IDS2017 | 98.9 | 98.3 | 99.2 | 98.8 |
| Autoencoder | UNSW-NB15 | 95.2 | 94.7 | 95.6 | 95.1 |
| Autoencoder | CSE-CIC-IDS2018 | 96.1 | 95.2 | 96.8 | 96.0 |
| Isolation Forest | CIC-IDS2017 | 93.2 | 92.1 | 93.8 | 92.9 |
| Isolation Forest | UNSW-NB15 | 91.7 | 90.9 | 92.2 | 91.5 |
| Isolation Forest | CSE-CIC-IDS2018 | 92.4 | 91.2 | 93.0 | 91.5 |

### 3.1 Cross-Dataset Performance Analysis

To quantify the robustness of the model in detecting zero-day attacks, we tested both the autoencoder and Isolation Forest on external datasets (UNSW-NB15

and CSE-CIC-IDS2018) without retraining. Both models generalized well despite the differences in the domain and feature distributions. The autoencoder achieved over 95% accuracy across all datasets, and the Isolation Forest also performed well with no significant degradation. This confirms the performance of the models in detecting unknown attacks in real-world environments. In addition, further detailed explanations or discussions are necessary regarding the selection process of key hyperparameters for both the Isolation Forest and autoencoder models.

**Table 5.** COMPARISON OF RECENT STUDIES ON ZERO-DAY ATTACK DETECTION

| **Reference** | **Dataset** | **Method** | **Accuracy (%)** |
|---|---|---|---|
| [21] | UNSW_NB15 | Hybrid Model | 93.42 |
| [18] | CICIoT2023 | Hybrid( CNN-LSTM) | 94.95 |
| [11] | ADFA | One class SVM | 97.40 |
| [7] | MNIST | CNN | 97.88 |
| [19] | IBM | CNN | 98.4 |
| **Proposed** | **CIC-IDS2017** | **Autoencoder** | **98.9** |

## 4 Conclusion

In this study, we introduce a deep learning-based framework for detecting zero-day attacks. By leveraging an autoencoder trained on benign traffic, the proposed system demonstrated exceptional performance, with 98.9% accuracy and a high F1 score, significantly outperforming the baseline IF model. The low false-positive rate and strong generalization of the model make it well suited for real-world cybersecurity applications.

In the future, we will focus on leveraging the good performance of our proposed unsupervised system. First, we explored the use of transformer-based models, such as the Anomaly Transformer, to better learn the temporal dependencies in network traffic. These models have the potential to learn long-range patterns, which can improve stealthy or slowly changing-attack detection. Second, we will attempt to include explainable AI (XAI) techniques, such as SHAP or LIME. We also aim to provide explanations regarding the decision-making process of the autoencoder to increase transparency and usability for cybersecurity analysts. This is especially crucial for building trust in automated anomaly-detection systems. Third, we will scale our evaluation to resource-constrained environments, such as IoT and edge computing, using techniques such as model pruning, quantization, and federated learning, to facilitate light- and privacy-friendly deployment.

Finally, we will examine domain adaptation techniques to improve performance in new traffic environments without full retraining, including techniques such as unsupervised fine-tuning, adversarial domain matching, and online learning to adapt to emerging threats in real time.

## References

1. Azeez, N.A., Osamagbe, I.I., Chinyere, I.C.: Performance evaluation of covolutional neural networks (cnns) and recurrent neural networks (rnns) for intrusion detection. Covenant Journal of Informatics and Communication Technology (2024)
2. Bandarupalli, G.: Efficient deep neural network for intrusion detection using cic-ids-2017 dataset. In: 2025 First International Conference on Advances in Computer Science, Electrical, Electronics, and Communication Technologies (CE2CT). pp. 476–480. IEEE (2025)
3. Dai, Z., Por, L.Y., Chen, Y.L., Yang, J., Ku, C.S., Alizadehsani, R., Pławiak, P.: An intrusion detection model to detect zero-day attacks in unseen data using machine learning. PloS one **19**(9), e0308469 (2024)
4. Edakkat Parambil, A.K.: A Hybrid Ensemble Model using XGBoost and AdaBoost to detect and distinguish zero-day attacks. Ph.D. thesis, Dublin, National College of Ireland (2023)
5. Google Threat Analysis Group: We're all in this together: A year in review of zero-days exploited in-the-wild in 2023 (Apr 2024), https://googleprojectzero.blogspot.com/2024/04/zero-day-2023-in-review.html, accessed: 2025-04-26
6. Gopalan, S.S., Ravikumar, D., Linekar, D., Raza, A., Hasib, M.: Balancing approaches towards ml for ids: A survey for the cse-cic ids dataset. In: 2020 International Conference on Communications, Signal Processing, and their Applications (ICCSPA). pp. 1–6. IEEE (2021)
7. Hassen, M., Chan, P.K.: Learning a neural-network-based representation for open set recognition. In: Proceedings of the 2020 SIAM International Conference on Data Mining. pp. 154–162. SIAM (2020)
8. Hindy, H., Atkinson, R., Tachtatzis, C., Colin, J.N., Bayne, E., Bellekens, X.: Utilising deep learning techniques for effective zero-day attack detection. Electronics **9**(10), 1684 (2020)
9. Ibrahim Hairab, B., Aslan, H.K., Elsayed, M.S., Jurcut, A.D., Azer, M.A.: Anomaly detection of zero-day attacks based on cnn and regularization techniques. Electronics **12**(3), 573 (2023)
10. Khan, Z.I., Afzal, M.M., Shamsi, K.N.: A comprehensive study on cic-ids2017 dataset for intrusion detection systems. Int Res J Adv Eng Hub **2**(02), 254–260 (2024)
11. Khraisat, A., Gondal, I., Vamplew, P., Kamruzzaman, J., Alazab, A.: Hybrid intrusion detection system based on the stacking ensemble of c5 decision tree classifier and one class support vector machine. Electronics **9**(1), 173 (2020)
12. Kim, J.Y., Bu, S.J., Cho, S.B.: Zero-day malware detection using transferred generative adversarial networks based on deep autoencoders. Information Sciences **460**, 83–102 (2018)
13. Kumar, V., Sinha, D.: A robust intelligent zero-day cyber-attack detection technique. Complex & Intelligent Systems **7**(5), 2211–2234 (2021)

14. Narmadha, S., Balaji, N.: Improved network anomaly detection system using optimized autoencoder- lstm. Expert Systems with Applications p. 126854 (2025)
15. Oyelakin, A., Ameen, A., Ogundele, T., Salau-Ibrahim, T., Abdulrauf, U., Olufadi, H., Ajiboye, I., Muhammad-Thani, S., et al.: Overview and exploratory analyses of cicids 2017 intrusion detection dataset. Journal of Systems Engineering and Information Technology (JOSEIT) **2**(2), 45–52 (2023)
16. Paganini, P.: Google threat intelligence group (gtig) tracked 75 actively exploited zero-day flaws in 2024 (2025), accessed: 2025-05-05
17. Rehman, M.U., Garima, Alasbali, N., Jwaid, A.: Ensemble machine learning-based intrusion detection system for securing the internet of things. In: 2024 International Conference on Engineering and Emerging Technologies (ICEET). pp. 01–09 (2024). https://doi.org/10.1109/ICEET65156.2024.10913890
18. Saurabh, K., Singh, U., Mishra, A., Vyas, R., Vyas, O.: Zero-shot based hybrid neural network system for enhancing zero-day attack detection. In: 2024 IEEE 21st India Council International Conference (INDICON). pp. 1–6. IEEE (2024)
19. Touré, A., Imine, Y., Semnont, A., Delot, T., Gallais, A.: A framework for detecting zero-day exploits in network flows. Computer Networks **248**, 110476 (2024)
20. Trunz, E., Weinmann, M., Merzbach, S., Klein, R.: Efficient structuring of the latent space for controllable data reconstruction and compression. Graphics and Visual Computing **7**, 200059 (2022)
21. Wu, Y., Hu, Y., Wang, J., Feng, M., Dong, A., Yang, Y.: An active learning framework using deep q-network for zero-day attack detection. Computers & Security **139**, 103713 (2024)
22. Zhao, J., Shetty, S., Pan, J.W.: Feature-based transfer learning for network security. In: MILCOM 2017 - 2017 IEEE Military Communications Conference (MILCOM). pp. 17–22 (2017). https://doi.org/10.1109/MILCOM.2017.8170749